

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Кафедра экономической информатики

Дисциплина: Программирование сетевых приложений

**Отчёт о выполнении
лабораторной работы №3**
Вариант №4

Выполнял:
Биза Г.А.
гр. 414302

Проверил:
Иномов С.Ф.

Минск 2026

Цель работы: научиться проектировать и реализовывать иерархии классов в Java с использованием абстрактных классов и интерфейсов для демонстрации принципов ООП.

Задачи:

Иерархия: Product → Food (Perishable), Electronics (Warranty), Clothes (Size).

Интерфейсы: Expirable, Discountable.

Дополнения:

- Учёт скидок и сроков годности;
- Фильтры: категория ∧ цена ∧ срок годности;
- Статистика: средняя цена, топ-3 дорогих товара;
- Журнал: изменение цен.

Основа. Добавить продукт питания. Добавить электронику. Добавить одежду. Просмотреть весь ассортимент. Удалить товар (админ).

Редактировать цену/гарантию. Поиск по названию. Фильтр по категории.

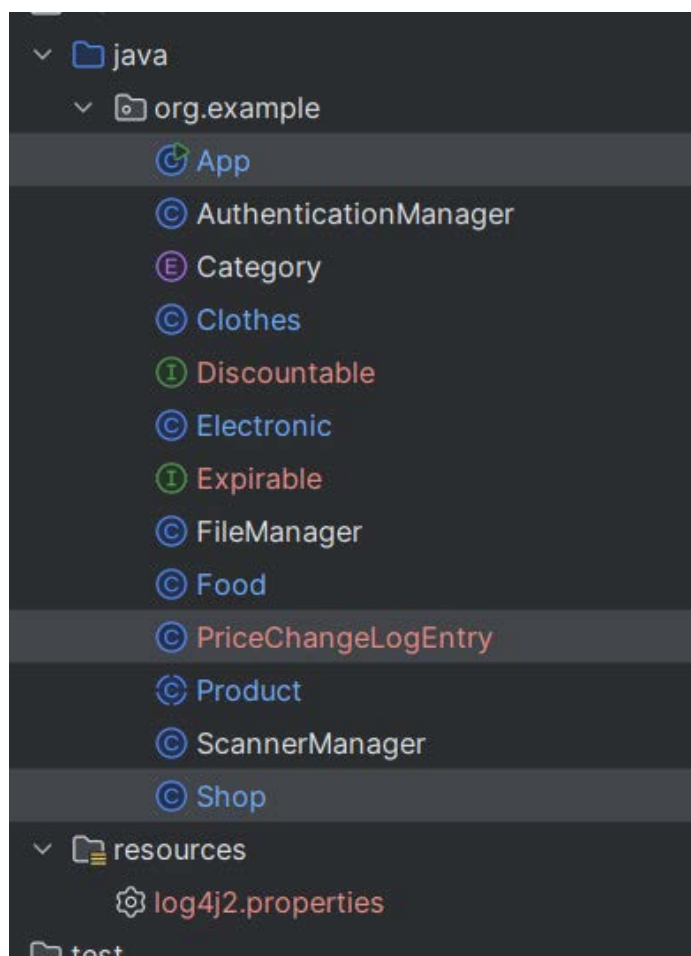
Фильтр по цене. Фильтр по диапазону цен. Сортировка по названию.

Сортировка по цене. Статистика по категориям. Сохранение данных.

Загрузка данных.

Ход выполнения работы

Структура проекта организована по примеру из методического пособия и выглядит следующим образом:



После этого нужно задать параметры для логирования в нашем консольном приложении. Через Log4j все события будут отображаться в консольном меню и записываться в отдельный файл, который называется app.log. В следующем фрагменте кода, показана настройка подключенной библиотеки:

```
rootLogger.level = info
rootLogger.appenderRefs = file
rootLogger.appenderRef.file.ref = File

appender.file.type = File
appender.file.name = File
appender.file.fileName = logs/app.log
appender.file.layout.type = PatternLayout
appender.file.layout.pattern = %d{HH:mm:ss.SSS} %-5level %logger{36} - %msg%n
```

Далее представлен полный листинг кода главного класса программы:

```
package org.example;

import java.time.LocalDate;
import java.util.List;
import java.util.Locale;
import java.util.Optional;

import static org.example.ScannerManager.*;

public class App
{
    public static void main( String[] args )
    {
        ScannerManager scan = new ScannerManager(System.in);
        while(true){
            System.out.println("""
                Выберите сценарий работы программы:
                1. Добавить продукт питания
                2. Добавить электронику
                3. Добавить одежду
                4. Просмотреть весь ассортимент
                5. Удалить товар
                6. Редактировать цену
                7. Редактировать гарантию (электроника)
                8. Установить скидку на товар (одежда/электроника)
                9. Поиск по названию
                10. Фильтр по категории + цена + срок годности
                11. Фильтр по цене (точная)
                12. Фильтр по диапазону цен
                13. Сортировка по названию
                14. Сортировка по цене
                15. Статистика по категориям и топ-3
                16. Журнал изменения цен
                17. Сохранение данных
                18. Загрузка данных
                0. Выход из программы
                """);

            switch (scan.scanRangeInt(0,18)){
                case 0 -> {
                    return;
                }
            }
        }
    }
}
```

```

case 1 -> {
    System.out.println("Введите название продукта питания:");
    String name = scan.scanString();
    System.out.println("Введите стоимость за единицу товара:");
    int price = scan.scanInt();
    System.out.println("Введите дату изготовления");
    LocalDate manufacturedDate = scan.scanLocaleDate();
    System.out.println("Введите количество дней который товар будет годен для потребления:");
    int someDays = scan.scanInt();
    System.out.println("Введите количество товара, который вы хотите добавить");
    int quantity = scan.scanInt();
    Shop.add(new Food(name, price, Category.FOOD, quantity, manufacturedDate, someDays));
}

case 2 ->{
    System.out.println("Введите название прибора, который хотите добавить:");
    String name = scan.scanString();
    System.out.println("Введите стоимость за единицу товара:");
    int price = scan.scanInt();
    System.out.println("Введите гарантийный срок товара в днях:");
    int someDays = scan.scanInt();
    System.out.println("Введите количество товара, который вы хотите добавить");
    int quantity = scan.scanInt();
    System.out.println("Введите скидку в процентах (0-100):");
    double discount = scan.scanRangeInt(0,100);
    Shop.add(new Electronic(name, price, Category.ELECTRONICS, quantity, someDays, discount));
}

case 3 ->{
    System.out.println("Введите название элемента одежды, который хотите добавить:");
    String name = scan.scanString();
    System.out.println("Введите стоимость за единицу товара:");
    int price = scan.scanRangeInt(0, 10000);
    System.out.println("Введите количество товара, который вы хотите добавить");
    int quantity = scan.scanInt();
    System.out.println("Введите название бренда, выпустившего данную одежду");
    String brandName = scan.scanString();
    System.out.println("Введите размер (XS, S, M, L, XL, XXL):");
    String size = scan.scanString();
    System.out.println("Введите скидку в процентах (0-100):");
    double discount = scan.scanRangeInt(0, 100);
    Shop.add(new Clothes(name, price, Category.CLOTHES, quantity, brandName, size, discount));
}

case 4 ->{
    List<Product> productList = Shop.getProductList();
    if(productList.isEmpty()){
        System.out.println("В магазине отсутствует товар");
    }else {
        productList.forEach(System.out::println);
    }
}

case 5 -> {
    System.out.println("Введите пароль!");
    String password = scan.scanString();
    if (AuthenticationManager.checkPass(password)) {
        List<Product> productList = Shop.getProductList();
        if (productList.isEmpty()) {
            System.out.println("В магазине нет товаров для удаления.");
            break;
        }
        System.out.println("Список товаров:");
    }
}

```

```

        for (int i = 0; i < productList.size(); i++) {
            System.out.println(i + ". " + productList.get(i).getName() + " категория " +
productList.get(i).getCategory());
        }
        System.out.println("Введите номер товара, который хотите удалить:");
        int choice = scan.scanInt();
        if (choice >= 0 && choice < productList.size()) {
            Shop.remove(productList.get(choice));
        } else {
            System.out.println("Вы ввели неверный номер товара");
        }
    } else {
        System.out.println("Вы ввели неправильный пароль");
    }
}
case 6->{ System.out.println("Введите пароль!");
String password = scan.scanString();
if (AuthenticationManager.checkPass(password)) {
    System.out.println("Список товаров:");
    List<Product> productList = Shop.getProductList();
    if (productList.isEmpty()) {
        System.out.println("В магазине нет товаров для изменения цены.");
        break;
    }
    for (int i = 0; i < productList.size(); i++) {
        System.out.println(i + ". " + productList.get(i).getName() + " категория " +
productList.get(i).getCategory());
    }
    System.out.println("Введите номер товара, цену которого хотите изменить:");
    int choice = scan.scanRangeInt(0,productList.size() - 1);
    System.out.println("Введите новую цену товара");
    int newPrice = scan.scanRangeInt(0,10000);
    Shop.changePrice(productList.get(choice), newPrice);
} else {
    System.out.println("Вы ввели неправильный пароль");
}
}

case 7 -> {
    System.out.println("Введите пароль!");
    String password = scan.scanString();
    if (AuthenticationManager.checkPass(password)) {
        List<Product> productList = Shop.getProductList();
        if(productList.isEmpty()){
            System.out.println("В магазине отсутствует товар");
            break;
        } else {
            for(Product product : productList){
                if(product instanceof Electronic electronic){
                    System.out.println("- " + product.getName() + " (гарантия: " + electronic.getGuaranteeTime() + "
дн.)");
                }
            }
        }
    }
    System.out.println("Введите название товара, гарантию которого хотите изменить:");
    String productName = scan.scanString();
    Optional<Product> optionalProduct = Shop.findFirstProductByName(productName);
    if(optionalProduct.isPresent()){
        Product product = optionalProduct.get();

        if(product instanceof Electronic electronic){
            System.out.println("Введите новый гарантийный срок:");

```

```

        int newGuarantee = scan.scanInt();
        electronic.setGuaranteeTime(newGuarantee);
        System.out.println("Гарантия успешно обновлена!");
    }else {
        System.out.println("Ошибка: товар " + productName + " не является электроникой");
    }
}
}
}
}
}
}
}
case 8 -> {
    System.out.println("Введите пароль:");
    String password = scan.scanString();
    if (AuthenticationManager.checkPass(password)) {
        System.out.println("Список товаров, поддерживающих скидку (одежда/электроника):");
        Shop.getProductList().stream()
            .filter(p -> p instanceof Clothes || p instanceof Electronic)
            .forEach(p -> System.out.println(p.getName() + " (" + p.getCategory() + ")"));
        System.out.println("Введите название товара:");
        String name = scan.scanString();
        Optional<Product> opt = Shop.findFirstProductByName(name);
        if (opt.isPresent()) {
            System.out.println("Введите новую скидку (0-100):");
            double discount = scan.scanRangeInt(0,100);
            Shop.setDiscount(opt.get(), discount);
        } else {
            System.out.println("Товар не найден.");
        }
    } else {
        System.out.println("Неверный пароль.");
    }
}
}

case 9 ->{
    System.out.println("Введите название товара, который хотите найти");
    String productName = scan.scanString();
    Shop.findProductsByName(productName).forEach(System.out::println);
}

case 10 -> {
    System.out.println("Выберите категорию (введите номер):\n1. Продукты питания\n2. Одежда\n3. Электроника\n0. Все категории");
    int catChoice = scan.scanRangeInt(0, 3);
    Category cat = switch (catChoice) {
        case 1 -> Category.FOOD;
        case 2 -> Category.CLOTHES;
        case 3 -> Category.ELECTRONICS;
        default -> null;
    };
    System.out.println("Введите минимальную цену (со скидкой):");
    int minPrice = scan.scanInt();
    System.out.println("Введите максимальную цену:");
    int maxPrice = scan.scanInt();if (minPrice > maxPrice) {
        int temp = minPrice;
        minPrice = maxPrice;
        maxPrice = temp;
        System.out.println("Минимальная цена больше максимальной, значения автоматически переставлены.");
    }
    System.out.println("Фильтр по сроку годности (1 – только просроченные, 2 – только свежие, 0 – без фильтра):");
    int expChoice = scan.scanRangeInt(0, 2);

```

```

        Boolean expiredStatus = switch (expChoice) {
            case 1 -> true;
            case 2 -> false;
            default -> null;
        };
        List<Product> filtered = Shop.filterProducts(cat, minPrice, maxPrice, expiredStatus);
        if (filtered.isEmpty()) {
            System.out.println("Товары не найдены.");
        } else {
            filtered.forEach(System.out::println);
        }
    }
    case 11->{
        System.out.println("Введите точную стоимость товара, который хотите найти:");
        int price = scan.scanInt();
        if(!Shop.getProductList().isEmpty()){
            Shop.filterByPrice(price).forEach(System.out::println);
        }else{
            System.out.println("В магазине отсутствуют товары!");
        }
    }
    case 12 ->{
        System.out.println("Введите минимальную стоимость товара, который хотите найти:");
        int minValue = scan.scanRangeInt(0, 10000);
        System.out.println("Введите максимальную стоимость товара, который хотите найти:");
        int maxValue = scan.scanRangeInt(0,10000);
        if (minValue > maxValue) {
            int temp = minValue;
            minValue = maxValue;
            maxValue = temp;
            System.out.println("Минимальная цена больше максимальной, значения автоматически пере-
ставлены.");
        }
        if(!Shop.getProductList().isEmpty()){
            Shop.filterByRangePrice(minValue,maxValue).forEach(System.out::println);
        }else{
            System.out.println("В магазине отсутствуют товары!");
        }
    }
    case 13 ->{
        Shop.sortByProductName();
    }
    case 14 ->{
        Shop.sortByProductPrice();
    }

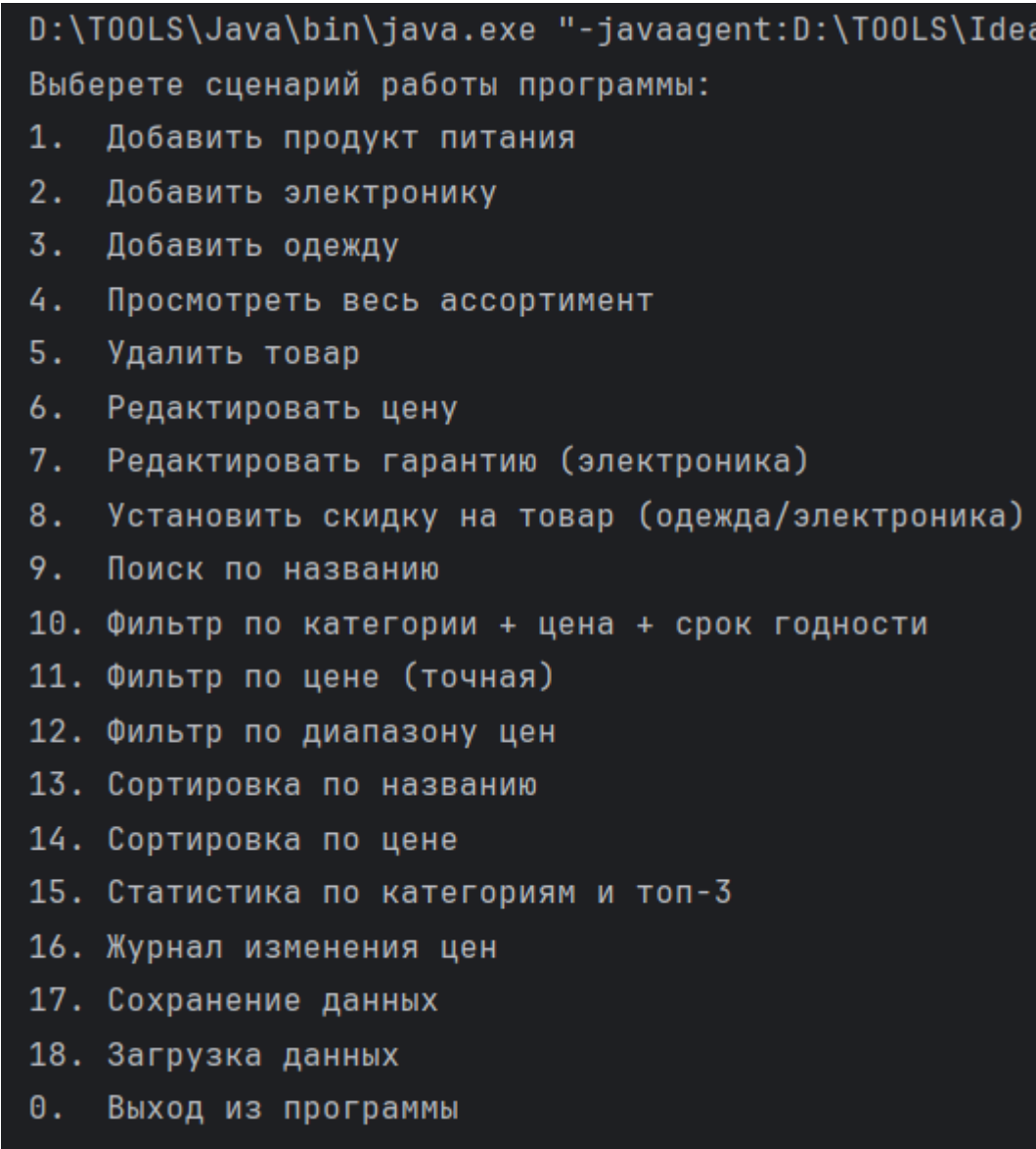
    case 15 ->{
        Shop.showStatistics();
    }

    case 16 -> Shop.showPriceLog();
    case 17->{
        FileManager.saveToFile(Shop.getProductList());
    }
    case 18->{
        List<Product> loaded = FileManager.loadFromFile();
        if (!loaded.isEmpty()) {
            Shop.getProductList().clear();
            Shop.getProductList().addAll(loaded);
        }
    }
}
}

```

```
}  
}  
}
```

На следующем рисунке представлен результат работы консольного приложения.



```
D:\TOOLS\Java\bin\java.exe "-javaagent:D:\TOOLS\Idea
Выберете сценарий работы программы:
1.  Добавить продукт питания
2.  Добавить электронику
3.  Добавить одежду
4.  Просмотреть весь ассортимент
5.  Удалить товар
6.  Редактировать цену
7.  Редактировать гарантию (электроника)
8.  Установить скидку на товар (одежда/электроника)
9.  Поиск по названию
10. Фильтр по категории + цена + срок годности
11. Фильтр по цене (точная)
12. Фильтр по диапазону цен
13. Сортировка по названию
14. Сортировка по цене
15. Статистика по категориям и топ-3
16. Журнал изменения цен
17. Сохранение данных
18. Загрузка данных
0.  Выход из программы
```

Рисунок 1 – результат работы консольного приложения

Вывод: в ходе выполнения лабораторной работы выполнена разработка консольного приложения на языке Java, которое помогает понять как устроены механизмы наследования и общие принципы ООП, особенно проработаны абстрактные типы данных.

Ответы на контрольные вопросы:

1. Что такое абстрактный класс в Java и для чего он используется?

Это класс, помеченный ключевым словом `abstract`. Он может содержать как полностью реализованные методы, так и абстрактные методы (без реализации). Используется для определения базового ("скелетного") поведения для будущих классов-наследников, задавая общий шаблон.

2. Можно ли создать объект абстрактного класса? Почему?

Нет, нельзя. Компилятор запрещает это, потому что абстрактный класс считается неполным (если в нем есть хотя бы один `abstract` метод, он не может быть вызван). Объект создается только от конкретного (полностью реализованного) класса-наследника

3. Чем абстрактный класс отличается от обычного класса?

Главное отличие — наличие ключевого слова `abstract` и возможность (но не обязательность) содержать абстрактные методы. Обычный класс (конкретный) может быть инстанцирован, абстрактный — нет.

4. Что такое абстрактный метод и как он объявляется?

Это метод, который объявлен без реализации (без тела). Он заканчивается точкой с запятой.

5. Можно ли в абстрактном классе реализовать обычные методы?

Да, обязательно. В этом и заключается его сила: можно реализовать общую логику для всех наследников, чтобы избежать дублирования кода, оставив изменяемые части на реализацию наследникам.

6. Можно ли в абстрактном классе объявлять конструкторы?

Да, можно. Они нужны для инициализации полей, объявленных в абстрактном классе. Вызываться они будут из конструкторов наследников через `super()`.

7. Что такое интерфейс в Java и для чего он нужен?

Интерфейс — это ссылочный тип, который может содержать только константы, абстрактные методы, `default`-методы и `static`-методы. Он задает **контракт** поведения. Любой класс, реализующий интерфейс, обязан предоставить реализацию всех его абстрактных методов. Нужен для обеспечения слабой связанности и возможности множественного "наследования" поведения.

8. Чем интерфейс отличается от абстрактного класса?

- **Состояние:** Абстрактный класс может иметь поля с любыми модификаторами, хранящие состояние. Интерфейс имеет только `public static final` константы.
- **Конструкторы:** У абстрактного класса есть конструкторы, у интерфейса — нет.

- **Наследование:** Класс может наследовать только один абстрактный класс (одиночное наследование), но может реализовать множество интерфейсов.
- **Модификаторы доступа:** В абстрактном классе методы могут иметь любые модификаторы (`private`, `protected`). В интерфейсе до Java 9 все методы были `public`. Сейчас могут быть `private`.

9. Можно ли класс наследовать несколько абстрактных классов?
 Нет. В Java запрещено множественное наследование классов (ромбовидная проблема). Класс может расширять (`extends`) только один класс (абстрактный или обычный).

10. Можно ли класс реализовать несколько интерфейсов?
 Да. Это одно из главных преимуществ интерфейсов. Класс может реализовать любое количество интерфейсов через запятую.

```
class Car implements Moveable, Refuelable, Comparable<Car> { ... }
```

11. Что такое множественная реализация интерфейсов и зачем она нужна?
 Это способность класса соответствовать нескольким контрактам одновременно. Нужна, чтобы описать объект, который может выполнять разные роли. Например, объект `Car` может быть и "Транспортным средством" (`Moveable`), и "Страховым случаем" (`Insurable`).

12. Что такое полиморфизм и как он проявляется при работе с интерфейсами?

Проявляется в том, что переменная типа интерфейса может ссылаться на объект любого класса, который этот интерфейс реализует.

```
Moveable m1 = new Car();
Moveable m2 = new Bicycle();
m1.move(); // Вызовется реализация из Car
m2.move(); // Вызовется реализация из Bicycle
```

Код работает с абстракцией `Moveable`, не зная конкретного типа транспорта.

13. Что такое ключевое слово `implements` и как оно используется?
 Ключевое слово `implements` используется классом, чтобы указать, что он берет на себя обязательства реализовать все абстрактные методы указанного интерфейса (или нескольких интерфейсов).

14. Что такое ключевое слово `extends` и как оно используется?
 Ключевое слово `extends` используется:

- Классом для указания родительского класса.
- Интерфейсом для расширения другого интерфейса.

15. Можно ли интерфейс наследовать от другого интерфейса?
Да. Интерфейс может расширять другой интерфейс (или несколько интерфейсов) с помощью ключевого слова `extends`.

```
interface ElectricVehicle extends Moveable, Rechargeable { }
```

16. Что такое default-методы в интерфейсах (с Java 8)?
Методы с ключевым словом `default` и реализацией по умолчанию. Позволяют добавлять новые методы в интерфейс, не ломая существующие классы-impleментаторы. Классы могут их переопределить или оставить реализацию по умолчанию.

17. Что такое static-методы в интерфейсах (с Java 8)?
Методы с ключевым словом `static` и реализацией. Принадлежат самому интерфейсу и вызываются через имя интерфейса (`MyInterface.staticMethod()`). Используются для вспомогательных утилитных функций, связанных с логикой интерфейса.

18. Можно ли в интерфейсе объявлять поля? Если да, то какие?
Да, можно. Но все поля интерфейса неявно являются `public static final`, то есть константами. Они обязаны быть инициализированы при объявлении.

19. Как работает аннотация @Override при реализации интерфейсов и абстрактных методов?

`@Override` проверяет, что метод действительно переопределяет (или реализует) метод из суперкласса или интерфейса. Если сигнатура метода не совпадает ни с одним методом родителя, компилятор выдаст ошибку. Это помогает избежать опечаток и поддерживать код в чистоте.

20. В каких случаях лучше использовать абстрактный класс, а в каких – интерфейс?

- **Абстрактный класс** лучше использовать, когда:
 - Есть тесно связанные классы с общей логикой (полями и реализованными методами), которую не хочется дублировать.
 - Нужно управлять доступом не только `public` (например, `protected` методы для наследников).
 - Нужно хранить нефинальное состояние объекта (поля, которые могут меняться).
- **Интерфейс** лучше использовать, когда:
 - Нужно задать контракт для несвязанных классов (например, умеющие летать: птица, самолет, дрон).
 - Классу нужно обладать несколькими ролями (множественное поведение).